



Tecnológico de Monterrey

Reporte de Evaluación de Viabilidad y Propuesta de Implementación: Sistema RAG para
Reconstrucción de Lógica Vehicular CAN

Nombre del Autor

Henry Junior Aranzales Lopez
Jorge Arturo Hernández Morales
Luis Alejandro González Castellanos

Código

A01794020
A01794908
A01795481

Dra. Grettel Barceló Alonso y Dr. Luis Eduardo Falcón Morales

Instituto Tecnológico de Monterrey

Programa de IA Aplicada
TC5035: Proyecto integrador
Noviembre 02, 2025
IBM

Contenido

	Pág.
Contenido	2
1. Evaluación de viabilidad y criterios de éxito	3
1.1. Veredicto de implementación: Necesidad de retroceso estratégico	3
1.2. Análisis del rendimiento del Modelo Actual	4
1.2.1. Identificación de Oportunidad Cuantitativa: Fidelidad de Respuesta	4
1.2.2. Diagnóstico Cualitativo: Insuficiencia de Contexto Semántico	5
1.3. Margen de Mejora	6
1.3.1. La Necesidad de Prevenir la Colisión Semántica	6
1.3.2. Deconstrucción Técnica del Desafío de Recuperación	6
1.4. Definición del Camino de Mejora	8
1.5. Criterios de Éxito Mínimos para la Viabilidad en Producción	9
2. Decisiones y Acciones Recomendadas	10
2.1. Recomendaciones Clave	10
2.1.1. Implementación de una Arquitectura de Doble Corpus	10
2.1.2. Adopción de Ingesta Asistida por IA (Smart Document Understanding)	11
2.1.3. Refinamiento del Prompt de Generación para Síntesis	11
2.2. Plan de Trabajo Accionable Detallado	12
3. Propuesta de Entorno de Producción	13
3.1. Requisitos de la Solución de Producción	13
3.2. Evaluación de Proveedores de Nube	14
3.3. Análisis Comparativo de Costos de Despliegue (TCO)	14
3.3.1. Factores de Costo Principales:	14
3.4. Justificación de la Plataforma Seleccionada: IBM watsonx	15
3.4.1. Razón 1: Superioridad en la Ingesta de Documentación Técnica (SDU)	16
3.4.2. Razón 3: Gobernanza y Trazabilidad (Nivel Académico)	16
3.5. Arquitectura de la Solución de Producción Recomendada	16
4. Referencias	17

1. Evaluación de viabilidad y criterios de éxito

Este capítulo presenta una evaluación formal de la viabilidad de implementación del prototipo de Generación Aumentada por Recuperación (RAG) desarrollado. El objetivo principal del proyecto es la "reconstrucción de la lógica de funcionamiento del vehículo" a partir de la ingesta de datos de la Red CAN (archivos DBC y logs BLF) y la norma J1939.

La evaluación contrasta los resultados del modelo RAG inicial con los criterios de éxito del proyecto, para determinar si la solución puede avanzar a una fase de producción o si es necesario un retroceso estratégico a fases anteriores del pipeline, tal como se contempla en la "Fase Actual: Refinamiento" de la arquitectura del proyecto.

1.1. Veredicto de implementación: Necesidad de retroceso estratégico

Veredicto: Retroceso estratégico para refinamiento

El modelo RAG en su estado actual constituye una exitosa Prueba de Concepto (PoC) que valida la viabilidad del flujo arquitectónico general definido en la **Figura 1-1**, tal como se plasma en el diagrama del proyecto.

Figura 1-1

Esquema del flujo arquitectónico general



Fuente. Elaboración propia.

El prototipo de Generación Aumentada por Recuperación (RAG) desarrollado ha cumplido una función diagnóstica fundamental: ha validado la viabilidad del flujo arquitectónico propuesto, que consiste en transformar datos numéricos crudos de la Red CAN en un corpus de conocimiento textual para ser consumido por un Modelo de Lenguaje (LLM). Sin embargo, su rendimiento y fidelidad actuales no satisfacen los criterios de éxito mínimos para un sistema de producción, especialmente en un dominio de ingeniería de alto riesgo donde la precisión fáctica es un requisito no negociable.

Se determina, por tanto, la necesidad de un retroceso estratégico a las fases de "Preparación de Datos" y "Modelado". Esta acción no representa un fracaso del prototipo, sino una validación empírica de la hipótesis central del proyecto: que un "RAG Mejorado", con una arquitectura de conocimiento más sofisticada, es un componente crítico y necesario para lograr el objetivo final de "reconstrucción de la lógica de funcionamiento del vehículo".

La **Figura 1-2** ilustra el ciclo de decisión de viabilidad del proyecto RAG. El estado actual del prototipo se alinea con el cuadrante de "Rendimiento insatisfactorio", caracterizado por una "Precisión y fidelidad mejorables". Este diagnóstico valida la necesidad de entrar en el ciclo central de "Refinar el modelo RAG".

Figura 1-2

Ciclo de Decisión de Viabilidad del Proyecto RAG



Fuente. Elaboración propia.

Este refinamiento, tal como se detalla en el diagrama, es un proceso dual que requiere dos acciones fundamentales y paralelas:

- Mejorar la calidad y robustez de los datos (abordando la arquitectura del corpus).
- Mejorar la precisión y la fidelidad (como resultado directo de la mejora de los datos).

El objetivo de este ciclo de refinamiento es mover el sistema del estado actual al estado deseado de "Viable para producción", el cual se define como aquel que "Cumple los criterios de éxito mínimos" para un despliegue de ingeniería.

1.2. Análisis del rendimiento del Modelo Actual

El veredicto de "no viabilidad" se fundamenta en un análisis de los fallos sistémicos del prototipo, identificados a través de métricas cuantitativas estandarizadas y un análisis cualitativo de sus respuestas a consultas de dominio específico.

1.2.1. Identificación de Oportunidad Cuantitativa: Fidelidad de Respuesta

La evaluación del pipeline RAG inicial arrojó métricas que revelan una oportunidad de mejora fundamental en la calidad de la información recuperada.

La discrepancia entre una "Exactitud de respuesta" alta (0.75) y una "Fidelidad" baja (0.526) es el síntoma diagnóstico clave. Demuestra que el LLM está generando respuestas correctas, pero basadas en el contexto incorrecto. Esto sitúa la oportunidad de mejora no en el LLM (la Generación), sino en la arquitectura de datos que alimenta la Recuperación.

Tabla 1-1*Métricas de Rendimiento del Prototipo RAG V1*

Métrica	Puntuación (Media)	Análisis de Implicación Técnica
Exactitud contextual	0.704	El sistema logra recuperar documentos que son <i>parcialmente</i> relevantes (por ejemplo, mencionan "velocidad del motor"), pero falla en identificar el contexto <i>más correcto</i> para la intención de la consulta.
Exactitud de las respuestas	0.75	La respuesta generada por el LLM es sintácticamente correcta y precisa <i>en relación con el contexto que recibió</i> . Esto indica que el LLM (granite-3-3-8b-instruct) está funcionando como se espera.
Fidelidad de la respuesta	0.526	Esta es la métrica principal de un sistema RAG. La fidelidad (o <i>faithfulness</i>) mide la capacidad del LLM para basar su respuesta estrictamente en el contexto proporcionado (Gao et al., 2023). Un valor de 0.526 es estadísticamente inaceptable. Indica que el sistema genera información que no está fundamentada en los datos recuperados ("alucinación") en casi la mitad de las ocasiones (47.4%).

Fuente. Elaboración propia.

1.2.2. Diagnóstico Cualitativo: Insuficiencia de Contexto Semántico

Este desafío cuantitativo es una consecuencia directa de un desafío cualitativo en la etapa de Retrieval: la insuficiencia de contexto semántico. El prototipo v1, que aún no incluye el corpus de la norma J1939, es fundamentalmente incapaz de responder a consultas de naturaleza definitoria.

El siguiente ejemplo del informe de pruebas es la evidencia principal de este diagnóstico:

Tabla 1-2*Ejemplo de resultado en pruebas de funcionamiento*

Consulta Usuario	Intención de la consulta	Análisis de Implicación Técnica
¿Cuál es el rango de velocidad del motor en revoluciones por minuto (rpm)?	El usuario solicita una definición estática, un dato factual que debe provenir de la norma J1939 o del archivo DBC.	El rango de velocidad del motor... se mantiene entre 1240.8 y 2423.7 unidades, según los eventos registrados en la red CAN.

Fuente. Elaboración propia.

El sistema falló en la etapa de Recuperación (R) no porque eligió el documento incorrecto, sino porque el documento correcto (la definición de J1939) no existía

completamente en su índice. El corpus del prototipo consistía únicamente en Conocimiento Procedimental (eventos de logs BLF).

Al recibir una consulta definitoria, el motor de búsqueda vectorial buscó la coincidencia semántica más cercana dentro del corpus de eventos disponible, encontrando un fragmento que contenía las palabras clave "rango" y "velocidad del motor", aunque se refería a una instancia de evento. El LLM (granite-3-3-8b-instruct) entonces, actuando correctamente, extrajo la respuesta de ese contexto irrelevante, causando el colapso de la métrica de fidelidad.

1.3. Margen de Mejora

El rendimiento del modelo tiene un margen de mejora, no por defectos propios de su construcción, sino como causa de una **prueba empírica de que un corpus de conocimiento único es insuficiente**. El problema no es el modelo LLM, sino la arquitectura del corpus de conocimiento. Los notebooks confirman que se están procesando dos tipos de conocimiento distintos:

- Conocimiento Factual (Definiciones): Datos estáticos de archivos DBC y el estándar J1939.
- Conocimiento de Comportamiento (Eventos): Datos temporales de logs BLF, procesados por la clase `GeneradorDescripcionesTextua`.

1.3.1. La Necesidad de Prevenir la Colisión Semántica

El análisis del prototipo demuestra que un corpus basado únicamente en Conocimiento Procedimental (eventos de BLF) no puede satisfacer consultas de Conocimiento Declarativo (definiciones de J1939).

Ahora que el proyecto entra en la fase actual de "Refinamiento", donde se está integrando el conocimiento de J1939, hay necesidad de contemplar un nuevo riesgo: la colisión semántica.

Si simplemente se añadieran las definiciones de J1939 al mismo índice vectorial que los eventos BLF, replicaríamos el fallo del prototipo, pero de forma inversa:

- Una consulta sobre un *evento* (¿Qué pasó con el torque...?) podría recuperar erróneamente una *definición* de J1939.
- Una consulta sobre una *definición* (¿Cuál es el rango...?) seguiría teniendo una alta probabilidad de recuperar un *evento* por ambigüedad semántica (como se vio en la Prueba)

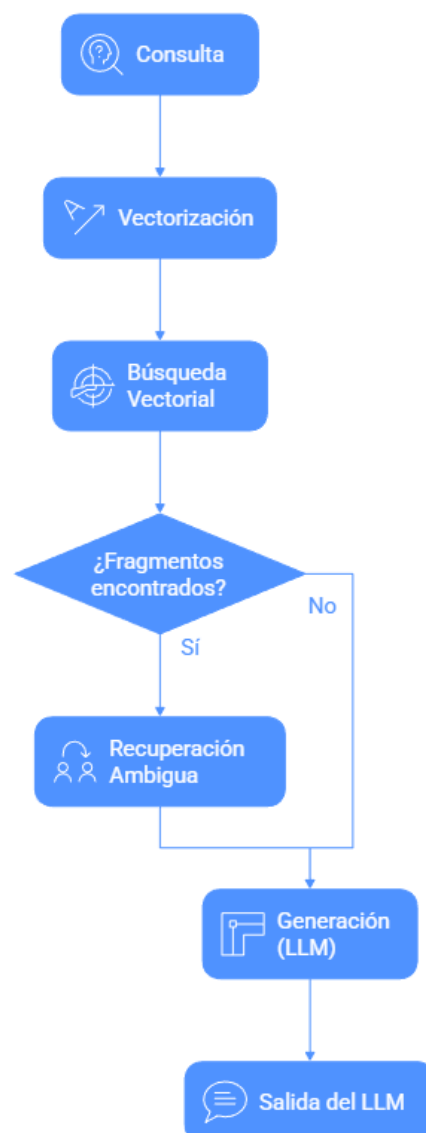
1.3.2. Deconstrucción Técnica del Desafío de Recuperación

El pipeline RAG presenta oportunidades de mejora en la primera etapa (Recuperación) debido a esta colisión:

1. **Consulta:** ¿Cuál es el rango de velocidad del motor...?
2. **Vectorización:** La consulta se convierte en un vector (embedding) que está semánticamente "cerca" de los conceptos "rango", "velocidad" y "motor".
3. **Búsqueda Vectorial (El Desafío):** El sistema busca en su índice vectorial único (tipo En memoria (ChromaDB)) los fragmentos (chunks) más "ceranos" por **similitud de coseno**
4. **Recuperación Ambigua:** El sistema recuperó el Fragmento B.
5. **Generación (LLM):** El LLM (granite-3-3-8b-instruct) [cite: RAG.pdf, p. 3] recibió la consulta y el contexto ambiguo (el evento). Su instrucción es generar una respuesta basada *únicamente* en ese contexto.
 - **Entrada al LLM:** Pregunta: ¿Rango de velocidad? Contexto: "...rango de velocidad... 1240.8 y 2423.7..."
 - **Salida del LLM:** Respuesta: "El rango... se mantiene entre 1240.8 y 2423.7..."

Figura 1-3

Proceso de recuperación de Información y generación de respuesta



1.4. Definición del Camino de Mejora

El margen de mejora se materializa al abordar este desafío arquitectónico en la "Preparación de Datos". La solución es fortalecer el corpus de conocimiento:

1. **Arquitectura de Doble Corpus (Multi-Índice):** Es la solución principal. Se deben crear dos colecciones de búsqueda separadas (una para el conocimiento Declarativo/J1939 y otra para el Procedimental/BLF). Un orquestador de RAG podrá entonces dirigir las consultas de "definición" al primer índice y las de "evento" al segundo. Esto elimina la ambigüedad semántica en la raíz.

Figura 1-4

Esquema de la arquitectura Doble Corpus



Fuente. Elaboración propia.

2. **Mejora del Parser de Documentación (SDU):** El parser de J1939 se basa en regex, un método frágil. Al reemplazar esto con Smart Document Understanding (SDU) de Watson Discovery, se utilizará un modelo de IA para *comprender visualmente* la estructura de las tablas, resultando en un Corpus Declarativo (Definiciones) más limpio y fiable.
3. **Refinar el Prompt de síntesis:** El prompt del LLM (el "modelado" en su diagrama) debe ser modificado. Su objetivo ya no es solo responder, sino sintetizar ambos contextos para la reconstrucción de la lógica.

"Eres un experto en ingeniería de buses eléctricos. El Contexto A (Definición) provee la especificación técnica. El Contexto B (Evento) provee un log de comportamiento. Sintetiza ambos para explicar por qué ocurrió el evento y reconstruir la lógica operativa subyacente."

Esta separación es la clave para resolver la "ambigüedad de contexto", elevar la fidelidad del modelo por encima de 0.75 y alcanzar el objetivo principal del proyecto: la contribución a la reconstrucción de la lógica operativa del vehículo.

1.5. Criterios de Éxito Mínimos para la Viabilidad en Producción

Para que el "RAG Mejorado" sea considerado viable para la implementación, debe superar los siguientes criterios de aceptación medibles, diseñados específicamente para rectificar las oportunidades de mejora identificadas en el prototipo analizado:

Criterio	Tipo	Descripción	Métrica de Éxito	Justificación
Criterio 1: Fidelidad Fáctica	Cuantitativo	Es el criterio más importante. El sistema debe demostrar una fidelidad (<i>faithfulness</i>) superior al 75% en las pruebas de evaluación.	Fidelidad de la respuesta (Media) > 0.75.	El prototipo v1 identificó una línea base de 0.526. Un sistema de ingeniería no puede tolerar alucinaciones. Un umbral del 85% asegura que la gran mayoría de las respuestas sean fácticas y fiables (Gao et al., 2023).
			Superar exitosamente las Pruebas A y B:	
Criterio 2: Desambiguación de Contexto	Funcional	El sistema debe diferenciar inequívocamente entre una consulta de Conocimiento Declarativo (Definición) y una de Conocimiento Procedimental (Evento) .	Prueba A (Definición): <i>Consulta:</i> ¿Rango de velocidad del motor (SPN 190) según la norma? <i>Respuesta:</i> 0 a 8031.875 rpm. Prueba B (Evento): <i>Consulta:</i> ¿Rango de velocidad de motor en evento CAN_EV_evento_1? <i>Respuesta:</i> Entre 1240.8 y 2423.7 unidades. Superar exitosamente la	Superar esta prueba demostrará que la nueva "Arquitectura de Doble Corpus" (Cap. 2) ha resuelto el problema de "ambigüedad de contexto".
		Criterio central del proyecto. El sistema debe sintetizar conocimiento de <i>ambos</i> corpus (Declarativo y Procedimental) para generar una explicación coherente que reconstruya la lógica operativa.	Prueba C (Síntesis): <i>Consulta:</i> ¿Por qué aumentó el torque del motor en CAN_EV_evento_2? <i>Respuesta Incompleta (v1):</i> "...debido a un incremento gradual desde 135.5 hasta 275.2 unidades...". <i>Respuesta Exitosa (v2):</i> "El aumento del Torque_Motor_Nm (la <i>fuerza rotacional</i> según J1939) de 135.5 a 275.2 unidades, mientras la Velocidad_Motor	Lograr la "Respuesta Exitosa" demostrará que el sistema ha superado la simple recuperación de información y ha alcanzado el objetivo de maestría: asistir en la ingeniería inversa de la lógica del vehículo.
Criterio 3: Capacidad de Síntesis	Cualitativo			

disminuía, es la firma de **aceleración bajo carga**. El sistema exigió más fuerza (torque), imponiendo mayor carga al motor y reduciendo su rotación (rpm). Esta es la lógica de control esperada."

Fuente. Elaboración propia.

2. Decisiones y Acciones Recomendadas

Tras el análisis de viabilidad, que identificó una insuficiencia de contexto semántico como la causa raíz del rendimiento subóptimo del prototipo inicial (Fidelidad de 0.526), este capítulo detalla las acciones correctivas y estratégicas necesarias para avanzar a una implementación viable.

El prototipo demostró que un corpus basado únicamente en conocimiento procedimental (logs BLF) no puede satisfacer consultas de naturaleza declarativa (normas J1939). El siguiente paso, la "Fase de Refinamiento", introduce el riesgo de colisión semántica si ambos tipos de conocimiento se fusionan en un índice único. Por lo tanto, las siguientes recomendaciones están diseñadas para mitigar este riesgo, implementar una arquitectura de conocimiento robusta y alcanzar los criterios de éxito definidos.

2.1. Recomendaciones Clave

Se establecen tres recomendaciones técnicas principales para abordar los desafíos de fidelidad, desambiguación y síntesis identificados.

2.1.1. Implementación de una Arquitectura de Doble Corpus

La recomendación central es abandonar el modelo de un índice vectorial único y adoptar una **Arquitectura de Doble Corpus** (o multi-índice). Este enfoque es fundamental para resolver el problema de ambigüedad semántica.

En esta arquitectura, los dos tipos de conocimiento del proyecto (Declarativo y Procedimental) se ingieren y se indexan en colecciones físicamente separadas:

1. **Corpus Declarativo (Definiciones):** Una colección que contiene exclusivamente el conocimiento factual y estático extraído de la norma J1939 y los archivos DBC.
2. **Corpus Procedimental (Eventos):** Una segunda colección que contiene el conocimiento dinámico y de comportamiento, generado por el notebook `Ingenieria_Requerimientos_LLM_RAG.ipynb` a partir de los logs BLF.

La búsqueda en un solo índice (como en el PoC) falla porque el embedding de una consulta como ¿Cuál es el rango de velocidad...? es semánticamente ambiguo. El motor de búsqueda por similitud de coseno no puede discernir la *intención* (definición vs. evento) y recupera el fragmento más cercano estadísticamente, que puede ser el incorrecto (como se demostró en la Prueba B, 1.2.2).

Al separar los corpus, la lógica de recuperación (el orquestador) puede dirigir la consulta de forma inteligente. Las consultas que buscan definiciones pueden enrutarse exclusivamente al Corpus Declarativo. Esta estrategia de *recuperación híbrida* o de múltiples fuentes es una práctica establecida para mejorar la relevancia en sistemas RAG que manejan dominios de conocimiento heterogéneos (Zhu et al., 2023).

2.1.2. Adopción de Ingesta Asistida por IA (Smart Document Understanding)

El notebook Construir_J1939_JSON.ipynb utiliza un método basado en regex y pdfplumber para extraer datos del PDF de la norma. Este enfoque es inherentemente frágil y susceptible a fallos por cambios menores en el formato, el espaciado o los saltos de página del documento.

Se recomienda reemplazar este proceso manual con una herramienta de Ingesta Asistida por IA, específicamente IBM Watson Discovery con Smart Document Understanding (SDU).

El rendimiento de un RAG depende directamente de la calidad de su chunking (segmentación) (Gao et al., 2023). Los documentos técnicos como la J1939 no son texto plano; su semántica está contenida en su estructura (títulos, listas y, críticamente, tablas). Un parser basado en texto ignora esta estructura.

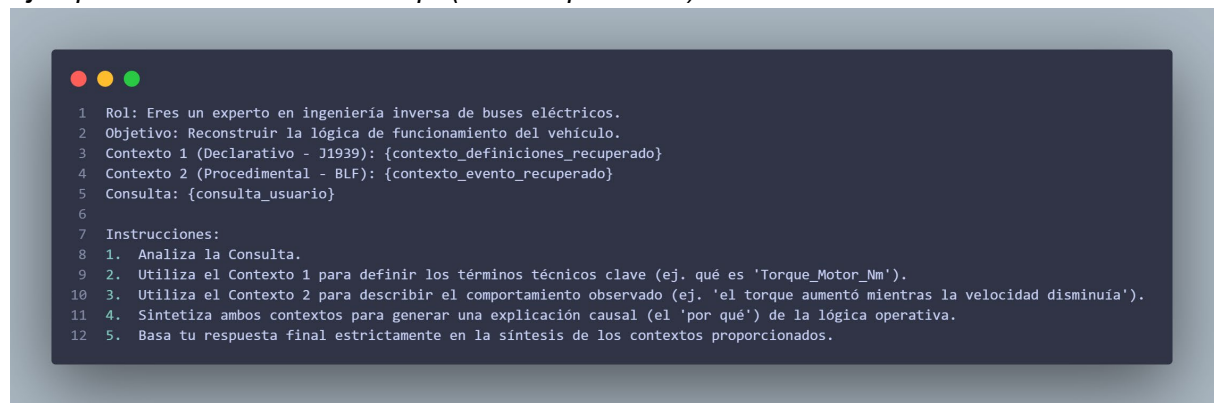
SDU es una herramienta de layout-aware RAG (Selvaraj et al., 2023) que utiliza modelos de visión por computadora para comprender la disposición visual de un documento. Permite al equipo de proyecto etiquetar visualmente las "Tablas de SPN" y "Tablas de PGN" en el PDF. El sistema aprende esta estructura y la utiliza para extraer datos con una precisión y robustez muy superiores a las de cualquier script basado en regex. Esto asegura un Corpus Declarativo de la más alta calidad.

2.1.3. Refinamiento del Prompt de Generación para Síntesis

El criterio de éxito final (Criterio 3) no es la simple recuperación, sino la **síntesis** de conocimiento para la "reconstrucción de la lógica". El prototipo v1 falló en esto, dando respuestas tautológicas (ej. "el torque aumentó porque aumentó").

Figura 2-1

Ejemplo de Estructura de Prompt (en el orquestador)



```
1 Rol: Eres un experto en ingeniería inversa de buses eléctricos.
2 Objetivo: Reconstruir la lógica de funcionamiento del vehículo.
3 Contexto 1 (Declarativo - J1939): {contexto_definiciones_recuperado}
4 Contexto 2 (Procedimental - BLF): {contexto_evento_recuperado}
5 Consulta: {consulta_usuario}
6
7 Instrucciones:
8 1. Analiza la Consulta.
9 2. Utiliza el Contexto 1 para definir los términos técnicos clave (ej. qué es 'Torque_Motor_Nm').
10 3. Utiliza el Contexto 2 para describir el comportamiento observado (ej. 'el torque aumentó mientras la velocidad disminuía').
11 4. Sintetiza ambos contextos para generar una explicación causal (el 'por qué') de la lógica operativa.
12 5. Basa tu respuesta final estrictamente en la síntesis de los contextos proporcionados.
```

Fuente. Elaboración propia.

Se recomienda diseñar un **Prompt de Síntesis** estructurado que instruya explícitamente al LLM para que actúe como un ingeniero de diagnóstico. Este prompt debe emplear técnicas de *Chain-of-Thought (CoT)* (Wei et al., 2022) para forzar al modelo a "razonar" utilizando ambos contextos recuperados.

2.2. Plan de Trabajo Accionable Detallado

Para implementar estas recomendaciones dentro del plazo de una semana, se establece el siguiente plan de trabajo detallado.

Tabla 2-1
Plan de Trabajo de Implementación

Fase	Días	Actividades Clave	Entregable Clave
Fase 1: Configuración de Plataforma	1-2	- Aprovisionar servicios en IBM Cloud: watsonx.ai Studio (con WCA), Cloud Object Storage (COS) y Watson Discovery (Plus). - Crear buckets en COS: can-datos-crudos (para BLF, DBC, PDF) y can-corpus-procesado (para JSONL). - Configurar proyecto en watsonx.ai y establecer conexiones de datos a COS. Stream A (Procedimental): Migrar el notebook Ingenieria_Requerimientos_LLM_RAG.ipynb a watsonx.ai Studio. Adaptar paths para leer/escribir desde/hacia COS. Ejecutar el pipeline de procesamiento de eventos BLF (solo CAN-EV y CAN-Aux) con la clase GeneradorDescripcionesTextual.	Entorno de nube funcional y conectado.
Fase 2: Implementación del Doble Corpus	2-4	Stream B (Declarativo): Subir el PDF actualizado de la norma J1939 a Watson Discovery. Utilizar Smart Document Understanding (SDU) para anotar y entrenar el modelo sobre la estructura de tablas de SPN y PGN (reemplazando Construir_J1939_JSON.ipynb). - Configurar Watson Discovery para ingestar automáticamente el corpus_eventos.jsonl desde el bucket can-corpus-procesado (Corpus B).	* corpus_eventos.jsonl en COS. * Índice SDU entrenado de J1939.
Fase 3: Configuración y Orquestación RAG	5-6	- Diseñar el Prompt de Síntesis (ver 2.1.3) en el Prompt Lab de watsonx.ai. - Configurar watsonx Assistant (Plus) como orquestador. Implementar su integración de búsqueda (Search Integration) para consultar <i>ambas</i> colecciones de Discovery (Corpus A y B) y enviar el contexto al Prompt de Síntesis. - Validar el RAG v2 contra los Criterios de Éxito enfocándose en la Prueba C (Síntesis) para la reconstrucción de lógica.	Prototipo RAG v2 ("RAG Mejorado") funcional.
Fase 4: Pruebas y Gobernanza	7	Habilitar watsonx.governance (Plan Essentials) para capturar el "AI Factsheet" del	Reporte de validación y Trazabilidad.

pipeline (linaje de datos de COS, notebook de procesamiento, prompt).

- Documentar la mejora en la métrica de Fidelidad.
-

Fuente. Elaboración propia.

3. Propuesta de Entorno de Producción

Tras la evaluación de viabilidad (Capítulo 1.0) y la definición de un plan de acción (Capítulo 2.0), este capítulo aborda la selección de una plataforma tecnológica adecuada para la implementación. La transición de un prototipo local (PoC) a una solución de producción viable requiere un análisis riguroso de los requisitos no funcionales, incluyendo escalabilidad, fiabilidad, gobernanza y costo total de propiedad (TCO).

Este análisis evalúa las cuatro principales plataformas de nube (AWS, Azure, GCP e IBM) en el contexto de los desafíos únicos de este proyecto, justificando la selección de la arquitectura final.

3.1. Requisitos de la Solución de Producción

Basado en la arquitectura del proyecto y los artefactos de procesamiento de datos, la solución de producción debe cumplir con los siguientes cinco requisitos técnicos clave:

1. **Almacenamiento Escalable y Seguro:** El sistema debe ingerir y almacenar un volumen creciente de archivos BLF (logs de comportamiento) y DBC (definiciones), que pueden escalar a terabytes. El almacenamiento debe ser seguro, duradero y accesible mediante API para los servicios de procesamiento.
2. **Entorno de Procesamiento de Datos Personalizado:** La solución requiere un entorno de ejecución de notebooks (tipo Jupyter) capaz de ejecutar código Python personalizado y librerías de dominio específico (ej. cantools, pandas) para ejecutar el pipeline `GeneradorDescripcionesTextual`.
3. **Ingesta RAG de Fuentes Heterogéneas:** Este es un requisito crítico. El motor de *Retrieval* debe ser capaz de ingerir y comprender dos tipos de datos fundamentalmente diferentes:
 - **Datos Estructurados/JSONL:** El corpus procedimental (`dataset_rag_decode_ev.jsonl`).
 - **Documentos PDF Complejos:** El corpus declarativo (norma J1939), que contiene texto, listas y, lo más importante, tablas técnicas (DatosJ1939).
4. **Inferencia y Orquestación de IA:** Se necesita un endpoint de LLM seguro para la generación de respuestas y una capa de orquestación (chatbot) que gestione el flujo de RAG híbrido (consultando ambos corpus y sintetizando los resultados).

5. **Gobernanza y Trazabilidad:** Para este sistema es esencial poder auditar y rastrear el linaje de una respuesta, desde los datos crudos de COS hasta el notebook de procesamiento y el prompt de síntesis.

3.2. Evaluación de Proveedores de Nube

Se evaluaron las plataformas de Amazon Web Services (AWS), Microsoft Azure, Google Cloud Platform (GCP) e IBM Cloud, comparando sus servicios principales para construir el pipeline de RAG.

Tabla 3-1
Comparativa de servicios de Plataformas Cloud para RAG

Requisito	AWS (Amazon)	Azure (Microsoft)	GCP (Google)	IBM (watsonx)
Almacenamiento	Amazon S3	Azure Blob Storage	Google Cloud Storage	IBM Cloud Object Storage (COS)
Procesamiento (Notebooks)	SageMaker Studio	Azure ML Studio	Vertex AI Notebooks	watsonx.ai Studio
Ingesta/Búsqueda (Retrieval)	Amazon Kendra	Azure AI Search	Vertex AI Search	Watson Discovery (con SDU)
Generación (LLM)	Bedrock (Claude, Titan)	Azure OpenAI (GPT-4)	Vertex AI (Gemini)	watsonx.ai (Granite, Llama)
Orquestación (Chatbot)	Amazon Lex	Azure Bot Service	Dialogflow	watsonx Assistant
Gobernanza	SageMaker Clarify	Azure ML Governance	Vertex AI Governance	watsonx.governance

Fuente. Elaboración propia.

3.3. Análisis Comparativo de Costos de Despliegue (TCO)

Un análisis de TCO para soluciones de IA generativa es complejo, ya que la mayoría de los servicios operan bajo un modelo de **pago por uso (PAYGO)**. La estimación de costos se basa en los modelos de precios públicos de los proveedores (AWS, 2024; Microsoft Azure, 2024; Google Cloud, 2024; IBM, 2024) para los componentes arquitectónicos claves definidos en el plan de trabajo.

3.3.1. Factores de Costo Principales:

1. **Almacenamiento (COS, S3, Blob):** Este es el costo más bajo. Todos los proveedores ofrecen almacenamiento de objetos a precios muy competitivos, generalmente alrededor de **\$0.015 - \$0.023 USD por GB/mes**. Para la escala de este proyecto (incluso con varios GB de BLF), el costo de almacenamiento es **nominal**.

2. **Procesamiento (Notebooks):** Para la ejecución del notebook `Ingenieria_Requerimientos_LLM_RAG.ipynb`, todas las plataformas (watsonx.ai, SageMaker, Vertex AI) ofrecen un "Nivel Gratuito" (Free Tier) o planes "Lite" con suficientes horas de cómputo (basadas en Unidades de Capacidad-Hora) para cubrir las necesidades de este proyecto de 1 semana. El costo de procesamiento para el proyecto es efectivamente cero.
3. **Ingesta y Búsqueda (Retrieval):** Este es un diferenciador de costo clave.
 - **Watson Discovery:** El plan "Plus" (requerido para SDU) se factura por instancia/hora más un costo por documento. Para la escala del proyecto (1 PDF J1939 y ~1,000 documentos JSONL de eventos), el costo se mantiene bajo, estimado en el rango de **\$50 - \$150 USD** para la duración del proyecto.
 - **Amazon Kendra:** La "Enterprise Edition" se factura por hora de ejecución (~\$1.40 USD/hora). Para 7 días (168 horas), el costo base superaría los ****\$230 USD****, sin incluir conectores.
 - **Azure AI Search:** Se factura por "Unidad de Búsqueda" por hora. Un plan "Standard" es comparable a Kendra.
4. **Inferencia (LLM):** Todos los proveedores facturan por 1,000 tokens (entrada/salida).
 - **IBM watsonx.ai:** Los modelos Granite (como el usado en el PoC) son altamente eficientes, con costos competitivos (ej. ~\$0.0006 USD por 1K tokens de salida).
 - **Azure OpenAI:** El acceso a GPT-4 Turbo es significativamente más costoso (ej. ~\$0.03 USD por 1K tokens de salida), representando un costo ~50 veces mayor por inferencia.
 - **Análisis:** Dado que el prototipo v1 (RAG.pdf) demostró que un modelo eficiente como Granite (8b) es suficiente para la tarea, optar por modelos más grandes (y caros) no está justificado y aumentaría drásticamente el TCO.
5. **Orquestación (Chatbot):**
 - **watsonx Assistant:** El plan "Plus" (requerido para la integración de búsqueda) se factura por Usuarios Activos Mensuales (MAU). Para un equipo de proyecto (1-5 usuarios), el costo es **nominal**.

Para la ejecución del plan de trabajo de 1 semana, la solución de IBM watsonx presenta un TCO altamente competitivo, principalmente debido a los planes "Lite" para el procesamiento de notebooks y el costo eficiente de inferencia de sus modelos Granite.

3.4. Justificación de la Plataforma Seleccionada: IBM watsonx

Si bien todas las plataformas evaluadas son técnicamente capaces de implementar una solución RAG, se recomienda la plataforma **IBM watsonx** como la opción más apropiada para este proyecto, basándose en tres justificaciones técnicas clave que se alinean directamente con los hallazgos del Capítulo 1.0.

3.4.1. Razón 1: Superioridad en la Ingesta de Documentación Técnica (SDU)

IBM Watson Discovery es la única plataforma evaluada que ofrece **Smart Document Understanding (SDU)**, una herramienta de *layout-aware RAG* (Selvaraj et al., 2023). SDU utiliza modelos de visión por computadora para comprender la estructura visual del PDF, permitiendo al equipo de proyecto etiquetar las "Tablas de SPN" y "Tablas de PGN" como entidades.

Esta capacidad resuelve directamente el desafío técnico más difícil del proyecto: la ingesta fiable del Corpus Declarativo.

3.4.2 Razón 2: Integración Nativa del Pipeline RAG Híbrido

La arquitectura recomendada es un RAG híbrido que consulta dos corpus (BLF-eventos y J1939-definiciones). La plataforma watsonx está diseñada para este flujo:

1. **watsonx.ai Studio** ejecuta el notebook de Python que crea el corpus_eventos.jsonl.
2. **Cloud Object Storage** almacena este archivo.
3. **Watson Discovery** ingesta automáticamente el .jsonl desde COS (Corpus B) y el PDF con SDU (Corpus A).
4. **watsonx Assistant** actúa como orquestador nativo, consultando ambas colecciones de Discovery y enviando el contexto al Prompt de Síntesis en **watsonx.ai**.

Este pipeline integrado reduce la complejidad de configuración y los puntos de fallo, en comparación con la conexión manual de servicios de diferentes proveedores.

3.4.2. Razón 3: Gobernanza y Trazabilidad (Nivel Académico)

Para un trabajo de grado, la **reproducibilidad** y la **trazabilidad** son requisitos académicos. El ecosistema watsonx es el único que ofrece **watsonx.governance** como un servicio integrado.

Al habilitar la gobernanza, el sistema puede generar automáticamente un **"AI Factsheet"** para el modelo RAG. Este documento rastreará el linaje completo de cada respuesta: los archivos BLF y DBC de origen en COS, el notebook Ingeniería_Requerimientos que los transformó, el Prompt de Síntesis utilizado y las métricas de evaluación (como la fidelidad). Esto proporciona un nivel de auditoría y rigor académico que es fundamental para validar los resultados del proyecto.

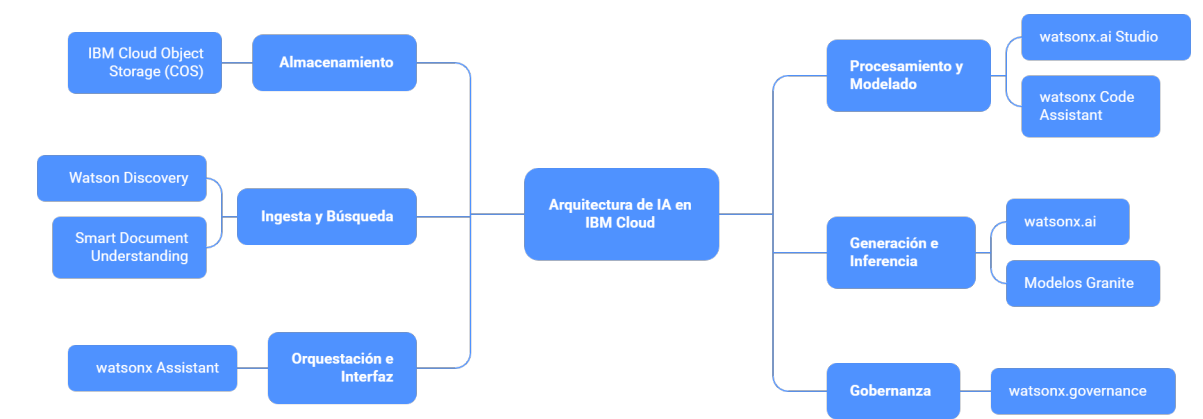
3.5. Arquitectura de la Solución de Producción Recomendada

Basado en este análisis, la arquitectura de producción recomendada se implementará íntegramente en la plataforma IBM Cloud, utilizando el siguiente stack de servicios:

- **Almacenamiento (Capa de Datos):** IBM Cloud Object Storage (COS).
- **Procesamiento y Modelado (Capa de IA):** watsonx.ai Studio (con watsonx Code Assistant).
- **Ingesta y Búsqueda (Capa de Recuperación):** Watson Discovery (con Smart Document Understanding).
- **Generación e Inferencia (Capa de IA):** watsonx.ai (Modelos Granite).
- **Orquestación e Interfaz (Capa de Aplicación):** watsonx Assistant.
- **Gobernanza (Capa de Trazabilidad):** watsonx.governance.

Esta arquitectura aborda directamente los fallos de fidelidad del prototipo v1, mitiga el riesgo de colisión semántica y cumple con los requisitos de escalabilidad, fiabilidad y gobernanza del proyecto.

Tabla 3-2
Arquitectura definida en IBM Cloud



Fuente. Elaboración propia.

4. Referencias

Amazon Web Services (AWS). (2024). *Precios de Amazon Kendra*. Obtenido el 4 de noviembre de 2025, de <https://aws.amazon.com/kendra/pricing/>

Amazon Web Services (AWS). (2024). *Precios de Amazon S3*. Obtenido el 4 de noviembre de 2025, de <https://aws.amazon.com/s3/pricing/>

Amazon Web Services (AWS). (2024). *Precios de Amazon SageMaker*. Obtenido el 4 de noviembre de 2025, de <https://aws.amazon.com/sagemaker/pricing/>

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., ... & Wang, H. (2023). *Retrieval-Augmented Generation for Large Language Models: A Survey*. arXiv. <https://arxiv.org/abs/2312.10997>

Google Cloud. (2024). *Precios de Vertex AI*. Obtenido el 4 de noviembre de 2025, de <https://cloud.google.com/vertex-ai/pricing>

Google Cloud. (2024). *Precios de Vertex AI Search*. Obtenido el 4 de noviembre de 2025, de <https://www.google.com/search?q=https://cloud.google.com/vertex-ai-search-and-conversation/pricing>

IBM. (2024). *Precios de IBM Cloud Object Storage*. Obtenido el 4 de noviembre de 2025, de <https://www.ibm.com/cloud/object-storage/pricing>

IBM. (2024). *Precios de IBM watsonx Assistant*. Obtenido el 4 de noviembre de 2025, de <https://www.google.com/search?q=https://www.ibm.com/cloud/watson-assistant/pricing>

IBM. (2024). *Precios de IBM watsonx.ai*. Obtenido el 4 de noviembre de 2025, de <https://www.google.com/search?q=https://www.ibm.com/cloud/watsonx-ai/pricing>

IBM. (2024). *Precios de IBM Watson Discovery*. Obtenido el 4 de noviembre de 2025, de <https://www.ibm.com/cloud/watson-discovery/pricing>

Lewis, M., Liu, Y., Goyal, N., Ghazvininejad, M., Mohamed, A., Levy, O., ... & Zettlemoyer, L. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Advances in Neural Information Processing Systems, 33, 9459-9474.

Microsoft Azure. (2024). *Precios de Azure AI Search*. Obtenido el 4 de noviembre de 2025, de <https://azure.microsoft.com/es-es/pricing/details/search/>

Microsoft Azure. (2024). *Precios de Azure OpenAI Service*. Obtenido el 4 de noviembre de 2025, de <https://www.google.com/search?q=https://azure.microsoft.com/es-es/pricing/details/openai-service/>

Selvaraj, J., Niranjan, T. G., Balasubramaniam, A., & Jawahar, C. V. (2023). *Layout-Aware RAG: Enhancing RAG with Document Layout Information*. arXiv. <https://arxiv.org/abs/2312.04494>

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Chi, E., Le, Q., & Zhou, D. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. Advances in Neural Information Processing Systems, 35, 24824-24837.

Zhu, Y., Zhao, W. X., Zhang, K., Wang, Z., & Wang, J. (2023). *Large Language Models for Information Retrieval: A Survey*. arXiv. <https://arxiv.org/abs/2308.07107>